

Lab 1: DataLab

备注：本报告使用 GPT6.1 辅助改善文字表述和公式表达。

P1

只需要将 1 左移到最高位，即可构造 0x80000000：

$$1 \ll 31 = 0x80000000.$$

P2

由于异或、与、取反均作用于比特位，因此可直接使用异或公式，并通过德摩根律消去或运算。以下用 $\oplus$ 、 $\wedge$ 、 $\vee$ 、 $\sim$ 分别表示按位异或、按位与、按位或、按位取反：

$$\begin{aligned} x \oplus y &= (x \wedge \sim y) \vee (\sim x \wedge y) \\ &= \sim [(x \wedge \sim y) \wedge \sim (\sim x \wedge y)]. \end{aligned}$$

P3

考虑分类讨论：若 $x \geq 0$ ，则 $(-x) \wedge 0 = 0$ ；若 $x < 0$ ，则 $(-x) \wedge (-1) = -x$ 。在本实验的补码和算术右移规则下， $-x = \sim x + 1$ ，且右移时高位填充原符号位，因此：

$$x \gg 31 = \begin{cases} 0, & x \geq 0, \\ -1, & x < 0. \end{cases}$$

故答案为：

$$\text{ans} = (\sim x + 1) \wedge (x \gg 31).$$

P4

发现字节编号 shift 为 0, 1, 2, 3 时，对应比特位分别为 0–7、8–15、16–23、24–31。由于 $\text{shift} \ll 3 = 8 \text{ shift}$ ，可以将对应字节右移到最低位，再通过掩码取得其数据：

$$\text{byte} = (x \gg (\text{shift} \ll 3)) \wedge 0xFF.$$

P5

逻辑右移需要在高位填充 0，因此不能简单地使用 $x \gg n$ 。先构造最高 n 位均为 0、其余位均为 1 的掩码 mask，即可消除算术右移后高位填充的 1：

$$\text{ans} = (x \gg n) \wedge \text{mask}.$$

P6

这道题的思路与 P4 类似：构造特定掩码取出 x 中的特定比特位，并通过或运算组合，难点在于构造掩码。此处通过移位运算构造 $\text{mask} = 0x0F0F0F0F$ ，从而逐字节取出高 4 位和低 4 位的数据，并交换它们的位置：

$$\text{ans} = ((x \gg 4) \wedge \text{mask}) \vee ((x \wedge \text{mask}) \ll 4).$$

P7

题干让我联想到提取最低位 1 的公式，但题目不允许直接使用负号。由补码关系 $-x = \sim x + 1$ ，得到：

$$\text{lowbit}(x) = x \wedge (-x) = x \wedge (\sim x + 1).$$

对 $\sim x$ 使用该公式，自然得到最低位 0 的掩码：

$$\text{lowbit}(\sim x) = (\sim x) \wedge (x + 1).$$

将这一掩码与 x 进行或运算，即可将最低位 0 置为 1：

$$x' = x \vee ((\sim x) \wedge (x + 1)) = x \vee (x + 1).$$

此时 x' 的最低位 0 就是原数的次低位 0，再提取一次即可：

$$\text{ans} = (\sim x') \wedge (x' + 1).$$

P8

题干要求 x 中有偶数个 1 时返回 1，有奇数个 1 时返回 0，因此想到将所有比特位异或后取布尔反。设最低位编号为 0，第 i 位为 b_i ，则：

$$\text{ans} = \neg \left(\bigoplus_{i=0}^{31} b_i \right).$$

只需依次将当前数与其右移 16, 8, 4, 2, 1 位后的数异或，便可将所有比特位的异或结果汇总到最低位。最终用 $!(x \& 1)$ 取布尔反即可，其中 $\neg$ 表示逻辑非。

P9

本题比较简单：原先想法为通过掩码保存低 n 位的数据，再左移到最高位，与 x 右移 n 位后的数据进行或运算拼接。随即发现右移后会出现 P5 中的符号扩展问题，并不等同于逻辑右移，因此先通过掩码保留右移后有效的高位数据，再与左移对应位数后的 x 拼接。需要注意，题目没有给出 $n < 32$ 的限制，因此先将移位数归一化：

$$n \leftarrow n \bmod 32.$$

对于非负整数 n ，这也可以通过 $n \wedge 31$ 实现；左移位数取 $(32 - n) \bmod 32$ ，以处理 $n = 0$ 的情况。

P10

思考后发现 2^n 的倍数实际上指的是“低 n 位均为 0 的数”，因此题干可转化为“找到距离 x 最近的低 n 位为 0 的数”。

记 $\text{mask}(x)$ 表示将 x 的低 n 位清零， b_i 为从低往高第 $i + 1$ 个比特位，则向下和向上舍入的候选数分别为：

$$L = \text{mask}(x), \quad U = L + 2^n.$$

两者除以 2^n 后的商相差 1，其中 $L/2^n$ 的奇偶性由 b_n 决定。定义布尔量 S 表示是否选择 U ，则：

$$\text{ans} = L + S \cdot 2^n = \text{mask}(x + S \cdot 2^n).$$

再定义布尔量 t 表示低 $n - 1$ 位是否不全为 0，即：

$$t = \bigvee_{i=0}^{n-2} b_i.$$

当 $n = 1$ 时没有这些低位，约定 $t = 0$ 。下面通过分类讨论确定 S ：

b_n	b_{n-1}	t	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

由真值表可得：

$$\begin{aligned}
 S &= (\neg b_n \wedge b_{n-1} \wedge t) \vee (b_n \wedge b_{n-1} \wedge \neg t) \vee (b_n \wedge b_{n-1} \wedge t) \\
 &= (b_n \wedge b_{n-1}) \vee (b_{n-1} \wedge t).
 \end{aligned}$$

令 $r = x - \text{mask}(x)$ ，即低 n 位表示的数值。发现超过中点的情况满足：

$$(b_{n-1} \wedge t) = 1 \iff r > 2^{n-1} \iff r + 2^{n-1} - 1 \geq 2^n.$$

恰好位于中点且应向上舍入的情况满足：

$$((b_{n-1} \wedge t) = 0) \wedge ((b_n \wedge b_{n-1}) = 1) \iff (r = 2^{n-1}) \wedge (b_n = 1).$$

因此，将 b_n 加入偏移量，就能统一这两种进位情况：

$$S = 1 \iff r + 2^{n-1} + b_n - 1 \geq 2^n.$$

由于上述加法结果在 $[0, 2^{n+1})$ 内，清零其低 n 位即可取得进位：

$$S \cdot 2^n = \text{mask}(r + 2^{n-1} + b_n - 1).$$

利用 $x = L + r$ ，且 L 已经是 2^n 的倍数，最终得到：

$$\begin{aligned}
 \text{ans} &= L + \text{mask}(r + 2^{n-1} + b_n - 1) \\
 &= \text{mask}(L + r + 2^{n-1} + b_n - 1) \\
 &= \text{mask}(x + 2^{n-1} + b_n - 1).
 \end{aligned}$$

P11

本题要求 x 、 y 的中点，若中点不是整数，则选择靠近第一个参数 x 的相邻整数。直接计算 $x + y$ 可能溢出，因此考虑从比特位拆分加法：两数同一位均为 1 时，对和贡献两份该位的权值；只有一个为 1 时，贡献一份。因此有 $x + y = 2(x \& y) + (x \oplus y)$ 。先将后一部分减半，再与前一部分相加，就能避免构造可能溢出的完整和。

在算术右移规则下，右移一位相当于除以 2 后向下取整，因此可先得到较小的候选整数：

$$L = \left\lfloor \frac{x + y}{2} \right\rfloor = (x \& y) + ((x \oplus y) \gg 1).$$

接下来判断是否需要将 L 加一。若两数同奇偶，则和为偶数，中点已经是整数；若两数奇偶不同，则和为奇数，中点位于 L 和 $L + 1$ 之间，此时 $x > y$ 应选择 $L + 1$ ， $x < y$ 应选择 L 。两数是否奇偶不同，只需检查异或结果的最低位。

最后还需要安全地判断 $x > y$ 。若二者异号，则 y 为负数时 $x > y$ ；若二者同号，则 $y - x$ 不会溢出，可以根据差值是否为负判断。将这一比较结果与奇偶判断结合，只有在和为奇数且 $x > y$ 时给 L 加一，即可得到靠近 x 的中点整数。

P12

本题需要判断 x 是否位于以 a 、 b 为端点的闭区间内，且两个端点的顺序不确定。考虑分别判断 $x < a$ 和 $x < b$ ：若两个判断结果不同，说明 x 不小于较小的端点，并且小于较大的端点；若结果相同，则说明 x 小于两个端点，或不小于两个端点。因此，可以通过两个比较结果的异或判断区间位置，再补上 $x = a$ 或 $x = b$ 的情况，从而包含两个端点，也能处理 $a = b$ 的情况。

难点在于不能直接使用比较运算符，也不能简单地根据 $x - a$ 的符号判断 $x < a$ ，因为异号的两个数相减可能溢出。因此需要按照符号分类讨论：若 x 与 a 异号，则 x 为负数时必有 $x < a$ ，否则 $x > a$ ；若二者同号，则差值不会溢出，可以根据 $x - a$ 的符号判断大小。对 x 和 b 的比较同理。

P13

本题要求计算 $5x$ ，若超出 32 位有符号整数的范围，则返回对应的上界或下界。由于不能使用乘法，可以通过加法依次计算 $2x = x + x$ 、 $4x = 2x + 2x$ 、 $5x = 4x + x$ 。关键在于检测中间步骤的溢出，不能只比较最终结果与 x 的符号，因为多次溢出后，最终结果的符号可能与原数相同。

在尚未发生溢出的情况下，每一步的两个加数都与 x 同号，因此首次溢出一定会使结果的符号翻转。分别对 x 与 $2x$ 、 $2x$ 与 $4x$ 、 $4x$ 与 $5x$ 进行异或，再将结果按位或，只要任意一步出现符号变化，合并

结果的符号位就为 1。将其算术右移 31 位，即可得到溢出时为 -1、未溢出时为 0 的掩码。最后，根据原数 x 的符号确定饱和值：非负数溢出时取 `INT_MAX`，负数溢出时取 `INT_MIN`；利用掩码在饱和值和正常计算结果之间选择，即可得到答案。

P14

本题需要判断三个整数的数学和是否超出 32 位有符号整数的范围，不能直接计算 $x + y + z$ ，因为中间结果可能溢出，也不能像 P13 那样逐步比较 $x, x + y, x + y + z$ 的符号变化。考虑将每个数拆成高位部分和低 2 位部分。在本实验的补码和算术右移规则下，有 $x = 4(x \gg 2) + (x \& 3)$ ，负数也满足这一关系。先将三个数分别右移 2 位后相加，得到高位和 `high`；再通过 `& 3` 取出各自的低 2 位，相加得到低位和 `low`。右移后每个数的范围缩小为原来的四分之一，因此三个高位部分相加不会溢出；低位和最多为 9，也不会溢出。

低位相加可能产生向高位的进位，因此还需要将 `low >> 2` 加到 `high` 上。记进位修正后的高位和为 h ，原数的数学和为 S ，则 $S = 4h + r$ ，其中 $0 \leq r \leq 3$ 。由于整数范围的两个边界恰好为 $\text{INT_MIN} = 4(-2^{29})$ 、 $\text{INT_MAX} = 4(2^{29} - 1) + 3$ ，因此无需还原完整的和，只需判断 h 的范围： $h \geq 2^{29}$ 时为正溢出，返回 1； $h < -2^{29}$ 时为负溢出，返回 -1；其余情况返回 0。

P15

本题需要构造浮点数乘以 $3/2$ 后的编码。考虑保留符号，只处理数值大小：先将有效数字乘以 3，再除以 2，并在最后统一舍入，避免中间舍入影响结果。乘以 3 可以拆成原数加上其两倍。NaN 和无穷大直接返回原编码，正零和负零保留原符号。

接下来需要区分非规格化数和规格化数。非规格化数没有隐含的首位 1，其数值为 $F \times 2^{-149}$ ，因此只需将小数域 F 乘以 $3/2$ 后舍入；若结果达到 2^{23} ，则转为最小指数的规格化数。对于规格化数，先补上隐含的首位 1，得到 24 位有效数字 M 。若 $3M < 2^{25}$ ，右移一位即可完成除以 2；否则结果需要更高的指数表示，应右移两位并将指数加一，以保持规格化形式。

右移时需要根据舍弃部分进行最近偶数舍入：余数小于半个单位则舍去，大于则进位，恰好等于时使保留结果的最低位为 0。若舍入再次产生超出 24 位的进位，则继续右移一位并增加指数。最后，指数达到 255 时返回带原符号的无穷大，否则按对应的规格化或非规格化格式组合符号、指数和小数域，得到结果。

P16

本题要求将浮点数舍入到最近的整数，并返回结果的浮点编码。考虑先按绝对值分类，保留符号后只判断大小：NaN 和无穷大原样返回；绝对值小于 0.5 时舍入为带原符号的零；恰好为 0.5 时也取零，因为

零是偶数；大于 0.5 且小于 1 时取带原符号的 1。单精度只有 24 位有效精度，因此绝对值达到 2^{23} 后，可表示的有限数已全部是整数，无需处理。

剩余情况的绝对值位于 $[1, 2^{23})$ ，需要确定哪些比特代表小数。设指数编码为 e ，实际指数为 $e - 127$ ，则小数部分占低 $s = 23 - (e - 127) = 150 - e$ 位。因此想到构造低 s 位为 1 的掩码，提取小数对应的余数，再清零这些位，得到向零截断的整数编码。将余数与 2^{s-1} 比较：不足则保留，超过则使整数的绝对值加一，恰好相等时根据整数最低位选择偶数。由于符号始终保留，这一判断同样适用于负数，也能保留舍入为零时的负号。

P17

本题需要构造整数转换为单精度浮点数后的编码。考虑先分离符号和大小：零直接返回零，负数记录符号，并用无符号数保存绝对值，以容纳 `INT_MIN` 的大小。随后找到绝对值中最高的 1，设其位置为 k ，则可写成 $1.f \times 2^k$ ，指数编码为 $k + 127$ 。

单精度只有 24 位有效精度，因此需将最高的 1 对齐到第 23 位（最低位编号为 0）。若 $k \leq 23$ ，左移补零即可精确表示；否则保留最高 24 位，并根据舍弃部分舍入：超过半个单位则进位，不足则舍去，恰好一半时使保留结果的最低位为 0。若舍入进位使有效数字变成 25 位，则右移一位并将指数加一。最后去掉隐含的首位 1，将剩余 23 位与符号、指数拼接，得到浮点编码。

P18

最直接的想法是逐位取出比特再相加，但这样需要较多操作。既然每个比特本身就表示 0 或 1，不妨把它看成这一位的计数，尝试同时合并多组相邻计数：先将每两位合并，再将每四位合并，逐步扩大分组，最终得到整个整数中 1 的个数。

首先考虑如何合并一对比特。需要将高位移到低位，与原来的低位相加，因此应当只保留每对比特的低位，避免不同组的数据混在一起。由此想到掩码应为重复的 01，即 `0x55555555`。分别取出低位和右移后的高位再相加，每组就得到 0、1 或 2，恰好可以用两位保存，不会向相邻组进位。

接下来，每两位已经代表一个计数，可以在每个四位块内合并两个计数。此时需要保留的是低两位，因此掩码自然变为重复的 0011，即 `0x33333333`。将高两位的计数右移对齐后与低两位相加，结果最多为 4，仍能放在四位块中。继续合并每个字节内的两个四位计数，所需掩码便是重复的 00001111，即 `0x0F0F0F0F`。一个字节最多有 8 个 1，只需要四位保存，因此可以先将原数与右移四位后的数相加，再用该掩码保留每个字节低四位的计数。其余四位中的数据会被清除，且这些局部和都不超过 8，不会产生跨四位块的进位。

按照同样的思路，后续似乎还需要用 `m8 = 0x00FF00FF` 和 `m16 = 0x0000FFFF` 清除合并后多余的局部计数。但最终只需取最低字节，因此可以反过来考察哪些数据会影响这一字节。设四个字节从高到低的计

数为 a, b, c, d ，与右移八位后的数相加后，各字节变为 $a, a + b, b + c, c + d$ 。此时 m_8 会清除第一、第三个字节中的 a 和 $b + c$ ，保留第二、第四个字节中的 $a + b$ 和 $c + d$ 。而下一轮右移十六位再相加，恰好只将第二个字节的 $a + b$ 加到最低字节的 $c + d$ 上；被 m_8 清除的两项均不会进入最低字节，因此省略 m_8 不影响总计数。同理， m_{16} 只清除高十六位，而最终提取最低字节时这些位本来就会被舍弃，因此也可以省略。各轮字节内的计数最多为 16 和 32，均小于 256，不会发生跨字节进位，保证了上述各字节之间互不干扰。最终最低字节得到 $a + b + c + d$ ，答案范围为 0 到 32，实际只需保留最低六位即可。

P19

本题需要将 32 位整数的所有比特位逆序，即原来的第 i 位移到第 $31 - i$ 位，最低位编号为 0。逐位提取再拼接需要较多操作，因此考虑分治交换：先交换高、低 16 位，再在每个 16 位块内交换高、低 8 位，随后依次在每个 8 位块、4 位块和 2 位块内交换两半。五轮交换后，每个比特位都到达逆序后的位置。

每轮交换都通过掩码选取各块的低半部分，将其左移到高半部分；同时将高半部分右移到低半部分，再用掩码保留需要的位，最后按位或组合。右移后使用掩码还可以清除算术右移产生的符号扩展位。各轮的移动位数和掩码如下：

移动位数	掩码	交换范围
16	0x0000FFFF	整个整数的高、低 16 位
8	0x00FF00FF	每个 16 位块的高、低 8 位
4	0x0F0F0F0F	每个字节的高、低 4 位
2	0x33333333	每个 4 位块的高、低 2 位
1	0x55555555	每对相邻比特位

掩码也不需要逐个独立构造：先由 $0xff$ 拼出 $0x0000FFFF$ ，再依次通过 $mask \wedge (mask \ll k)$ 更新，其中 k 为 8、4、2、1。左移后的掩码与原掩码在重叠区域均为 1，异或后这些区域变为 0，从而形成下一轮所需的交替分块掩码。这样可以复用掩码，在题目规定的操作次数内完成逆序。